



PROBLEM SESSION 7
Concurrency and Thread

คณะผู้จัดทำ

นัธวัฒน์	ปริมสิริคุณาวุฒิ	67070501027
วิศิษฐ์	สุวรรณเนา	67070501042
พลวิษฐ์	วัฒนเหมรัตน์	67070501067

เสนอ

ผศ. ราชวิช สโรชวิกสิต

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

CPE333 ระบบปฏิบัติการ (Operating Systems)

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ภาคเรียนที่ 1 ปีการศึกษา 2569

Problem 1: การทำงานหลาย Thread และ Race Condition

Step 1: การทำงานและจุดที่เกิด Race Condition ใน PS7.c

วัตถุประสงค์ของการทดลองคือศึกษาผลของการเข้าถึง **shared state** พร้อมกันผ่าน **POSIX Threads** และสังเกต **Race Condition** จากการปรับค่า **stock** โดยไม่มี **synchronization** โปรแกรม ps7.c เป็นการจำลองจำนวนสินค้าใน **stock** โดยเริ่มต้น 10,000 ชิ้น แต่ละเธรดทำซ้ำ 10,000 รอบ โดยจะเพิ่มสินค้า 1 ชิ้น และลดสินค้าลง 1 ชิ้นในแต่ละรอบ ดังนั้นค่าที่คาดหวังหลังทุกเธรดทำงานเสร็จคือ 10,000 ไม่ว่าจะใช้กี่เธรดก็ตาม ในฟังก์ชัน worker() จะเรียก change_stock(+1) และ change_stock(-1); ส่วนใน main() สร้างเธรดด้วย pthread_create() และรอให้ครบด้วย pthread_join() ก่อนแสดงผล

จุดที่เกิด **race condition** อยู่ใน change_stock() เพราะการอัปเดต **stock** เป็นชุดคำสั่งแบบ **read-modify-write** ไม่ใช่ **atomic operation** ได้แก่ (1) อ่าน **stock** ไปเก็บใน snapshot (2) คำนวณ snapshot += amount และ (3) เขียน **stock** = snapshot กลับ หากอีกเธรดแทรกเข้ามาระหว่างการอ่านกับการเขียน ค่าที่อ่านไว้จะล้าสมัยและอาจทับผลของเธรดอื่น เมื่อกำหนด yield=1 โปรแกรมเรียก sched_yield() ก่อนเขียนค่ากลับเพื่อเพิ่มโอกาสให้ scheduler เลือกเธรดอื่นเข้ามาคั่นกลาง แต่ sched_yield() ไม่ได้บังคับว่าจะเกิด context switch ทุกครั้ง

Source code

```
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>

#define INITIAL_STOCK 10000
#define ITERATIONS 10000

int stock = INITIAL_STOCK;
int force_yield = 0;

void change_stock(int amount) {
    int snapshot = stock;
    snapshot += amount;
    if (force_yield) {
        sched_yield();
    }
    stock = snapshot;
}

void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < ITERATIONS; i++) {
        change_stock(+1);
        change_stock(-1);
    }
    return NULL;
}
```

```

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <threads> <yield: 0|1>\n", argv[0]);
        return 1;
    }
    int thread_count = atoi(argv[1]);
    force_yield = atoi(argv[2]);
    if (thread_count < 1 || (force_yield != 0 && force_yield != 1)) {
        fprintf(stderr, "Invalid arguments\n");
        return 1;
    }
    pthread_t *threads = malloc(sizeof(*threads) * thread_count);
    if (threads == NULL) {
        perror("malloc");
        return 1;
    }
    for (int i = 0; i < thread_count; i++) {
        if (pthread_create(&threads[i], NULL, worker, NULL) != 0) {
            fprintf(stderr, "pthread_create failed\n");
            free(threads);
            return 1;
        }
    }
    for (int i = 0; i < thread_count; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("threads=%d yield=%d expected=%d actual=%d\n",
        thread_count, force_yield, INITIAL_STOCK, stock);
    free(threads);
    return 0;
}

```

Step 2: การคอมไพล์และกำหนดรูปแบบการทดลอง

หลังจากสร้างโค้ดดังกล่าวบน Linux VM ใน UTM แล้ว จึงคอมไพล์ด้วยคำสั่ง `gcc -O0 -Wall -Wextra -pthread -o ps7 ps7.c` จากนั้นบันทึกผลของแต่ละ Task ด้วย tee ดังนี้:

```

Task 1: ./ps7 1 0 | tee task1.txt
Task 2: for i in $(seq 1 10); do ./ps7 4 0; done | tee task2.txt
Task 3: for i in $(seq 1 10); do ./ps7 4 1; done | tee task3.txt

```

อาร์กิวเมนต์ตัวแรกคือจำนวน Thread และอาร์กิวเมนต์ตัวที่สองคือ 0 (ไม่เรียก `sched_yield()`) หรือ 1 (เรียก `sched_yield()`)

เพื่อให้การเปรียบเทียบทั้งสาม Task ยุติธรรม โปรแกรมใช้ `INITIAL_STOCK = 10,000` และ `ITERATIONS = 10,000` เหมือนกันทุก Task โดยเปลี่ยนเฉพาะจำนวน Thread และการเปิดหรือปิด `force_yield` ดังนั้นความแตกต่างของผลลัพธ์จึงสะท้อนผลจาก `concurrency` และช่วงเวลา queuing ที่ `scheduler` เข้ามาแทรกการทำงานเป็นหลัก

Step 3: ผลการทดลอง Task 1–3

จากการรัน เราได้ผลลัพธ์ในแต่ละ Task ดังนี้

Task 1 — 1 เธรด, **yield=0: expected=10000, actual=10000**

```
madmax@max-linux:~/ps7-0s$ gcc -O0 -Wall -Wextra -pthread -o ps7 ps7.c
madmax@max-linux:~/ps7-0s$ ./ps7 1 0 | tee task1.txt
threads=1 yield=0 expected=10000 actual=10000
madmax@max-linux:~/ps7-0s$
```

รูปที่ 1: ผลลัพธ์ของ Task 1 — Single Thread

Task 2 — 4 เธรด, **yield=0: ผลจากการรัน 10 ครั้งคือ 10000, 10000, 10000, 10000, 9999, 10002, 10000, 10001, 10000, 9999**

```
madmax@max-linux:~/ps7-0s$ for i in $(seq 1 10); do ./ps7 4 0; done | tee task2.txt
threads=4 yield=0 expected=10000 actual=10000
threads=4 yield=0 expected=10000 actual=10000
threads=4 yield=0 expected=10000 actual=10000
threads=4 yield=0 expected=10000 actual=10000
threads=4 yield=0 expected=10000 actual=9999
threads=4 yield=0 expected=10000 actual=10002
threads=4 yield=0 expected=10000 actual=10000
threads=4 yield=0 expected=10000 actual=10001
threads=4 yield=0 expected=10000 actual=10000
threads=4 yield=0 expected=10000 actual=9999
madmax@max-linux:~/ps7-0s$
```

รูปที่ 2: ผลลัพธ์ของ Task 2 — 4 Threads โดยไม่เรียก sched_yield()

Task 3 — 4 เธรด, **yield=1: ผลจากการรัน 10 ครั้งคือ 10000, 10000, 10000, 10000, 10001, 10001, 10000, 10001, 10001, 10000**

```
madmax@max-linux:~/ps7-0s$ for i in $(seq 1 10); do ./ps7 4 1; done | tee task3.txt
threads=4 yield=1 expected=10000 actual=10000
threads=4 yield=1 expected=10000 actual=10000
threads=4 yield=1 expected=10000 actual=10000
threads=4 yield=1 expected=10000 actual=10000
threads=4 yield=1 expected=10000 actual=10001
threads=4 yield=1 expected=10000 actual=10001
threads=4 yield=1 expected=10000 actual=10000
threads=4 yield=1 expected=10000 actual=10001
threads=4 yield=1 expected=10000 actual=10001
threads=4 yield=1 expected=10000 actual=10000
madmax@max-linux:~/ps7-0s$
```

รูปที่ 3: ผลลัพธ์ของ Task 3 — 4 Threads เมื่อใช้ sched_yield()

Step 4: วิเคราะห์ Race Condition และ Non-deterministic Result

ผลที่สังเกตได้คือ Task 1 ได้ 10000 เพราะมีเพียงเธรดเดียวที่แก้ไข **stock** ส่วน Task 2 พบ 9999, 10001 และ 10002 แม้ไม่ได้เรียก **sched_yield()** เพราะ **scheduler** สามารถสลับเธรดได้เอง Task 3 พบทั้ง 10000 และ 10001 เพราะการ **yield** เพิ่มโอกาสให้เกิด **interleaving** ใกล้เคียงเขียนกลับ แต่ไม่ได้บังคับว่าจะเกิด **context switch** หรือทำให้ผลผิดทุกครั้ง ดังนั้นค่า 10000 ในบาง run เป็นเพียง observed result ของ **interleaving** ที่บังเอิญไม่ทำให้เกิด **lost update** ไม่ใช่หลักฐานว่าโปรแกรมถูกต้องโดยไม่มี **synchronization; correctness** ต้องพิจารณาการรับประกันของโปรแกรม ไม่ใช่ผลจาก run เดียว

ตัวอย่างของ **lost update** คือเมื่อ **stock** เริ่มที่ 10,000 แล้ว Thread A และ Thread B อ่านค่า 10,000 พร้อมกัน ทั้งคู่คำนวณ +1 และเขียน 10,001 กลับลง **stock** ผลสุดท้ายจึงเป็น 10,001 แทนที่จะเป็น 10,002 เพราะการเขียนครั้งหลังทับผลของอีกเธรด หากเหตุการณ์ **interleaving** ลักษณะเดียวกันเกิดกับการลดค่า ผลลัพธ์สุดท้ายก็อาจสูงหรือต่ำกว่า 10,000 ได้ตามลำดับการอ่านและการเขียนที่เกิดขึ้นจริง

ที่สำคัญ การที่บาง run ยังได้ **expected=10000** ไม่ได้แปลว่า **race condition** หายไป เพราะอาจเป็นลำดับการทำงานที่บังเอิญไม่ทำให้เกิด **lost update** ในรอบนั้น ในโปรแกรม C/Pthreads นี้ หลายเธรดเข้าถึงออบเจกต์ **stock** ที่ไม่ใช่ atomic object โดยมีอย่างน้อยหนึ่งการเขียนและไม่มี **synchronization** จึงเกิด **data race**; ตาม **C memory model** พฤติกรรมนี้เป็น **undefined behavior** ทำให้มาตรฐานภาษาไม่รับประกันผลลัพธ์ที่แน่นอน ความแตกต่างของ **scheduler** จำนวน CPU core และ timing อาจทำให้ลำดับ **interleaving** เปลี่ยนไป ดังนั้นค่าจาก run ใด run หนึ่งไม่ใช่หลักฐานว่าโปรแกรมถูกต้อง

สรุปผลการทดลอง

Task 1 — Single Thread: **1 Thread, yield=0, expected=10000, actual=10000**

Task 2 — Multiple Threads: **4 Threads, yield=0**, รัน 10 ครั้ง พบ actual อยู่ในช่วง **9999–10002**

Task 3 — เรียก **sched_yield()** ระหว่างการอัปเดต: **4 Threads, yield=1**, รัน 10 ครั้ง พบ actual อยู่ในช่วง **10000–10001**

ผลการทดลองแสดงให้เห็นว่าเมื่อมีเพียง **1 Thread** โปรแกรมให้ผลตรงกับค่าที่คาดหวังคือ 10,000 แต่เมื่อหลาย Thread เข้าถึงและแก้ไขตัวแปร **stock** ร่วมกันโดยไม่มี **synchronization** ผลลัพธ์สามารถเปลี่ยนไปในแต่ละ run ได้จากการ interleave ของขั้นตอน **read-modify-write** การเรียก **sched_yield()** ระหว่างการอัปเดตช่วยเพิ่มโอกาสให้ Thread อื่นเข้ามาทำงานคั่นกลาง แต่ไม่ได้รับประกันว่าจะเกิด **context switch** ทุกครั้ง ดังนั้นผลอาจถูกต้องในบาง run และผิดในบาง run การทดลองนี้จึงแสดงผลของ **race condition** ต่อ **shared variable** ได้ตามวัตถุประสงค์ของ Problem Session 7 และชี้ให้เห็นว่าการใช้งานจริงควรป้องกัน **critical section** ด้วย **mutex** หรือ **atomic operation**

ข้อจำกัดของการทดลอง

ค่าที่ได้จาก Task 2 และ Task 3 ไม่ควรถูกตีความว่าเป็นช่วงผลลัพธ์ที่ระบบจะต้องให้เสมอ เพราะ **data race** ในภาษา C ทำให้มาตรฐานภาษาไม่รับประกันผลลัพธ์ และรายละเอียดของ **scheduler** จำนวน CPU core และ timing ของระบบสามารถเปลี่ยนรูปแบบ **interleaving** ได้ จุดสำคัญของการทดลองจึงอยู่ที่การแสดงให้เห็นว่าค่าของ **shared variable** ไม่สามารถคาดการณ์ได้อย่างแน่นอนเมื่อไม่มี **synchronization**

แนวทางป้องกันในระบบจริง

หากต้องการให้การอัปเดต **stock** ถูกต้องสม่ำเสมอ ควรกำหนด **critical section** ครอบงำช่วง **read-modify-write** แล้วป้องกันด้วย **pthread_mutex_t** หรือใช้ **atomic operation** ที่เหมาะสม อย่างไรก็ตามใน Problem Session นี้จึงไม่ใช้ **synchronization** เพื่อให้สามารถสังเกต **race condition** ได้โดยตรง